

Instrumentos Computacionales - Stata

Maestría en Economía

Santiago Cerutti

UNLP

Marzo 2026

Contenido - Clase 2

1. Macros
2. Procesos repetitivos (loops)
3. Funciones
4. Conversión de tipos de variables
5. Bysort
6. Unión de bases de datos
7. Transformación de bases
8. Trabajo práctico

Contenido - Clase 2

1. Macros
2. Procesos repetitivos (loops)
3. Funciones
4. Conversión de tipos de variables
5. Bysort
6. Unión de bases de datos
7. Transformación de bases
8. Trabajo práctico

Locales

Las macro locales permiten almacenar números y palabras para luego utilizarlos en distintas operaciones

```
* Para ALMACENAR valores (siempre usar el =)
local ejemplo_a = 1

* Para ALMACENAR letras debemos usar las comillas de texto:
local ejemplo_b = "uno"
```

Locales

Para utilizar el contenido de una macro local, encerramos su nombre entre comillas especiales:

```
* Para UTILIZAR el contenido:  
* Comilla de apertura: ` (alt + 96)  
* Comilla de cierre: ' (alt + 39)
```

```
display `ejemplo_a'
```

```
display "`ejemplo_b'"
```

Locales

- Las macro locales suelen utilizarse para almacenar resultados que se encuentren en la memoria temporal de STATA por haber ejecutado previamente algún otro comando.

Locales

- Las macro locales suelen utilizarse para almacenar resultados que se encuentren en la memoria temporal de STATA por haber ejecutado previamente algún otro comando.
- Por ejemplo, luego de ejecutar el comando **summarize**, se guardan temporalmente algunos estadísticos hasta que se realice otra estimación que los sobre-escriba.
- Con el comando **return list** podemos observar los valores almacenados, por ejemplo:
 - **r(mean)** contiene el promedio
 - **r(sd)** contiene el desvío estándar
 - **r(min)** contiene el mínimo

Locales

Por lo tanto, si queremos guardar en una local el promedio de la variable price

```
* Calculamos el promedio de price
summarize price

* Guardamos el resultado en una macro local
local precio_promedio = r(mean)

* Para ver el contenido de la macro local creada:
display `precio_promedio'
```


Locales

Resultado:

```
. summarize price
```

Variable	Obs	Mean	Std. dev.	Min	Max
price	74	6165.257	2949.496	3291	15906

```
. local precio_promedio = r(mean)
```

```
. display `precio_promedio'  
6165.2568
```

Pregunta

¿Sería lo mismo usar $(price > 6164.257)$ que $(price > 'precio_promedio')$ en una condición lógica?

Locales

- Como su nombre lo indica, las macro locales son locales al **contexto** en el que se las define.
- Si se definen dentro de un **do-file**, se pueden usar únicamente dentro de ese entorno. Cuando el do-file termina de correr, esas macros desaparecen.
- Si se definen en la **ventana de comandos**, se pueden usar únicamente dentro de esa sesión. Cuando se cierra Stata, esas macros desaparecen.
- Si ya existe una macro local con el mismo nombre, se **sobre-escribe** su contenido sin generar ningún tipo de advertencia.

Tempfiles

- De forma similar a la que almacenamos valores o texto temporalmente con las macros locales, es posible almacenar temporalmente una base de datos.
- **tempfile** asigna nombres a las macros locales especificadas que pueden usarse como nombres para archivos temporales.
- Al finalizar el programa o el archivo .do, se borran todos los conjuntos de datos creados con estos nombres asignados.

Tempfiles

Para **guardar** una base de datos temporalmente:

```
tempfile base_a  
save `base_a', replace
```

Para **cargar** la base de datos temporalmente:

```
use `base_a', clear
```

Globales

- Al igual que las macros locales, permiten almacenar valores o caracteres para luego utilizarlos en distintas operaciones.
- Pero, a diferencia de las locales, estas macros **no se borran** cuando el do-file termina de ejecutarse.

Macros

Globales

Para **definir** una macro global

```
global ejemplo_a = 1  
global ejemplo_b = "uno"
```

Para **utilizar** el contenido usamos de prefijo \$

```
display $ejemplo_a  
  
* Recomendación: usar {} para evitar problemas de encadenamiento de macros  
display ${ejemplo_b}
```

Contenido - Clase 2

1. Macros
2. Procesos repetitivos (loops)
3. Funciones
4. Conversión de tipos de variables
5. Bysort
6. Unión de bases de datos
7. Transformación de bases
8. Trabajo práctico

Los comandos para hacer bucles en STATA son:

- **foreach**: para iterar sobre elementos de una lista.
- **forvalues**: para iterar sobre un rango de valores.
- **while**: para iterar mientras se cumpla una condición.

Procesos repetitivos

Ejemplo 1

Hacer un tabulado de 5 variables cuyos nombres son:
v1, v2, v3, v4 y v5

```
* Sin bucle  
tabulate v1  
tabulate v2  
tabulate v3  
tabulate v4  
tabulate v5
```

Procesos repetitivos

Ejemplo 1

```
* Con bucle foreach
foreach x in v1 v2 v3 v4 v5 {
    tabulate `x'
}
```

En cada iteración **x** se irá reemplazando por cada una de las variables listadas luego del **in**, y ejecutará todas las operaciones que se incluyen entre las dos llaves.

Procesos repetitivos

Ejemplo 1

```
* Con bucle foreach y local para el listado de variables
local variables = "v1 v2 v3 v4 v5"

foreach x of local variables {
    tabulate `x'
}
```

En cada iteración **x** se irá reemplazando por cada una de las variables listadas dentro de la local **variables**, y ejecutará todas las operaciones que se incluyen entre las dos llaves.

Ejemplo 2

Almacenar los promedios de las variables **price**, **trunk** y **turn** en macros locales.

```
* Sin bucle
summarize price
local promedio_price = r(mean)

summarize trunk
local promedio_trunk = r(mean)

summarize turn
local promedio_turn = r(mean)
```

Procesos repetitivos

Ejemplo 2

```
* Usando foreach
foreach x of varlist price trunk turn {
    summ `x'
    local promedio_`x' = r(mean)
}
```

Pregunta

¿Y si quisiéramos que las locales con los promedios se llamaran: **prom1**, **prom2**, **prom3**?

Procesos repetitivos

Foreach

Sintaxis general

```
foreach lname { in | of listtype } list {  
    comando referido a `lname'  
}
```

* Variantes:

```
foreach lname in lista {  
foreach lname of local nombre_local {  
foreach lname of global nombre_global {  
foreach lname of numlist lista_numérica {  
foreach lname of varlist lista_variables {
```

Forvalues

Sintaxis general

```
forvalues i = rango {  
    comando referido a `i'  
}
```

Rangos más utilizados:

- n / N : i toma los valores desde n hasta N , con incrementos de 1
- $n (\delta) N$: i toma los valores desde n hasta N , con incrementos de δ

Procesos repetitivos

Ejemplo 1

```
* Con bucle forvalues
forvalues i = 1/5 {
    tabulate v`i'
}
```

En cada iteración **i** se irá reemplazando por cada número del 1 al 5 , completando el nombre de la variable.

Pregunta

¿Qué particularidad se da en este ejemplo que nos permite crear un bucle tanto con **forvalues** como con **foreach**?

Procesos repetitivos

Bucles anidados

Además, es posible anidar procesos repetitivos, lo que nos permite iterar a través de elementos de más de una lista o rango. Ejemplo:

```
foreach x of varlist price trunk turn {  
    summ `x'  
    foreach y in sd mean p50 {  
        local `y'_`x' = r(`y')  
    }  
}
```

Pregunta

¿Qué resultado obtendremos al ejecutar este código?

Procesos repetitivos

Bucles anidados

Es muy importante respetar la indentación para mejorar la legibilidad del código, aunque no es estrictamente necesaria para el correcto funcionamiento del mismo.

```
foreach x of varlist price trunk turn {  
  summ `x'  
  foreach y in sd mean p50 {  
    local `y'_`x' = r(`y')  
  }  
}
```

Procesos repetitivos

Bucles anidados

También es posible comentar el código dentro de un bucle anidado.

```
foreach x of varlist price trunk turn {  
    * Calculamos estadísticos de la variable `x'  
    summ `x'  
  
    foreach y in sd mean p50 {  
        * Guardamos los resultados en macros locales  
        local `y'_`x' = r(`y')  
    }  
}
```

Contenido - Clase 2

1. Macros
2. Procesos repetitivos (loops)
- 3. Funciones**
4. Conversión de tipos de variables
5. Bysort
6. Unión de bases de datos
7. Transformación de bases
8. Trabajo práctico

- Es posible crear o modificar variables utilizando funciones.
- La mayoría de las funciones requieren que le especifiquemos uno o más argumentos.
- Los argumentos se detallan dentro de los paréntesis de la función y se diferencian uno de otro mediante el uso de una coma.

```
* Ejemplo función ln(x):  
gen log_income = ln(income)
```

```
* Ejemplo función substr(s , n1 , n2):  
gen primera_letra = substr(string , 1 , 1)
```

Las funciones **NO** son comandos y suelen utilizarse en el espacio de la sintaxis donde imputamos las expresiones

```
comando [varlist] [=exp] [if exp] [in rango] [ , opciones]
```

Funciones

Para buscar los tipos de funciones disponibles podemos usar:

```
help function
```

Type of function	See help
Date and time	<code>datetime functions</code>
Mathematical	<code>mathematical functions</code>
Matrix	<code>matrix functions</code>
Programming	<code>programming functions</code>
Random-number	<code>random-number functions</code>
Selecting time spans	<code>time-series functions</code>
Statistical	<code>statistical functions</code>
String	<code>string functions</code>
Trigonometric	<code>trigonometric functions</code>

Funciones

Algunos ejemplos de transformaciones que podemos realizar mediante funciones de texto:

A

Original	Nueva
la Plata	LA PLATA
berisso	BERISSO
La matanza	LA MATANZA
tandil	TANDIL

B

Original	Nueva
Cod-1	1
Cod-2	2
Cod-6	6
Cod-9	9

C

Original	Nueva
Calle 12 1088	1088
Belgrano 13	13
Perú 2250	2250
Alberdi 4840	4840

Caso A

Debemos convertir a mayúsculas todos los caracteres de una variable existente, podemos utilizar la función:

```
upper(s)
```

Caso A

Debemos convertir a mayúsculas todos los caracteres de una variable existente, podemos utilizar la función:

```
upper(s)
```

Descripción: Devuelve la variante en mayúscula de la cadena de texto *s*.

```
* Ejemplo con la base auto  
gen make_2 = upper(make)
```

Caso B

Necesitamos extraer de la variable original solo una parte:

```
substr(s , n1 , n2)
```

Caso B

Necesitamos extraer de la variable original solo una parte:

```
substr(s , n1 , n2)
```

Descripción: Devuelve una subcadena de la cadena de texto *s*, comenzando en la posición ***n1*** y con una longitud de ***n2*** caracteres.

Si ***n1* < 0** ***n1*** se interpreta como la distancia desde el final de la cadena.
Si ***n2* = .** se devuelve la parte restante de la cadena.

Caso B

Necesitamos extraer de la variable original solo una parte:

```
substr(s, n1, n2)
```

Descripción: Devuelve una subcadena de la cadena de texto *s*, comenzando en la posición ***n1*** y con una longitud de ***n2*** caracteres.

Si ***n1* < 0** ***n1*** se interpreta como la distancia desde el final de la cadena.
Si ***n2* = .** se devuelve la parte restante de la cadena.

```
* Ejemplo con la base auto  
generate marca = substr(make_2, 1, 3)
```

Caso C

Necesitamos identificar la segunda palabra de la variable, esto podemos hacerlo con:

```
word(s , n)
```

Caso C

Necesitamos identificar la segunda palabra de la variable, esto podemos hacerlo con:

```
word(s, n)
```

Descripción: Devuelve la *n*-ésima palabra de la cadena de texto *s*.

```
* Ejemplos
display word("Este es un ejemplo", 4)
generate marca_2 = word(make_2, 1)
```

También es posible usar funciones dentro de otras funciones. En estos casos lo más simple es interpretar desde adentro hacia afuera

También es posible usar funciones dentro de otras funciones. En estos casos lo más simple es interpretar desde adentro hacia afuera

```
----- Función 1 -----  
display substr( word("Hola Mundo" , 2) , 1 , 2)
```

También es posible usar funciones dentro de otras funciones. En estos casos lo más simple es interpretar desde adentro hacia afuera

```
----- Función 1 -----  
display substr( word("Hola Mundo" , 2) , 1 , 2)
```

```
----- Resultado 1 ----  
display substr(      "Mundo"      , 1 , 2)  
-----Función 2-----
```

Funciones

También es posible usar funciones dentro de otras funciones. En estos casos lo más simple es interpretar desde adentro hacia afuera

```
----- Función 1 -----  
display substr( word("Hola Mundo" , 2) , 1 , 2)
```

```
----- Resultado 1 ----  
display substr(      "Mundo"      , 1 , 2)  
----- Función 2 -----
```

Resultado final:

Mu

Otras funciones

- El comando **egen** nos permite acceder a funciones adicionales.
- A diferencia del comando **generate**, el uso de una función aquí no es opcional sino que es obligatorio.
- **Recomendación:** Ver al menos una vez la lista completa de funciones disponibles con **egen** haciendo **help egen**.

Sintaxis general del comando:

```
egen newvar = funcion(argumentos) [if] [in] [ , opciones]
```

Contenido - Clase 2

1. Macros
2. Procesos repetitivos (loops)
3. Funciones
4. Conversión de tipos de variables
5. Bysort
6. Unión de bases de datos
7. Transformación de bases
8. Trabajo práctico

Conversión de tipos de variables

De texto a numéricas

Puede que tengamos una variable de texto que en realidad contenga solo números (como sucede al resolver el caso C) pero queramos transformarla a numérica.



Obs	original	nueva_str	nueva_num
1	Calle 10	10	10
2	Avenida 13	13	13
3	Calle 63	63	63
4	Avenida 7	7	7

De texto a numéricas

Para ello disponemos del comando **destring**, al que debemos especificarle la opción **gen()** o la opción **replace** para que nos genere una nueva variable o reemplace la variable con el contenido numérico, respectivamente.

Sintaxis general del comando:

```
destring varlist , {generate(newvarlist)|replace} [destring_options]
```

Ejemplo:

```
destring nueva_str , gen(nueva_num)
```

De texto a numéricas

Ejemplo:

```
destring nueva_str , gen(nueva_num)
```

Sin embargo, si existe algún carácter no numérico el comando solo funcionará agregando la opción **force**, pero convertirá automáticamente en missing estos valores.

De numéricas a texto

Para realizar el proceso inverso podemos utilizar el comando **tostring**, al que también debemos especificarle la opciones **gen()** o la opción **replace**.

```
tostring varname , gen(nueva_str)
```

De texto a numéricas

Pueden presentarse otros casos donde tengamos una variable de texto a partir de la cual queramos crear una variable idéntica pero de formato numérico que contenga los nombres en etiquetas de valor.

marca_num[1]				1
	make	marca	marca_...	
1	AMC Concord	AMC	AMC	
2	AMC Pacer	AMC	AMC	
3	AMC Spirit	AMC	AMC	
4	Buick Century	BUICK	BUICK	
5	Buick Electra	BUICK	BUICK	
6	Buick LeSabre	BUICK	BUICK	
7	Buick Opel	BUICK	BUICK	
8	Buick Regal	BUICK	BUICK	
9	Buick Riviera	BUICK	BUICK	
10	Buick Skylark	BUICK	BUICK	
11	Cad. Deville	CAD.	CAD.	
12	Cad. Eldorado	CAD.	CAD.	
13	Cad. Seville	CAD.	CAD.	
14	Chev. Chevette	CHEV.	CHEV.	
15	Chev. Impala	CHEV.	CHEV.	

De texto a numéricas

Alternativa 1: Usando el comando **encode**, cuya sintaxis es:

```
encode varname , gen(newvarname) generate(newvar) [label(name) noextend]
```

Este comando crea una nueva variable (newvar) basada en aquella variable de tipo string (varname), usando como etiquetas los valores de esta última.

```
* Ejemplo  
encode marca , gen(marca_num)
```

De texto a numéricas

Alternativa 2: Usando el comando **egen**, junto con la función **group()**.

```
* Ejemplo
egen marca = group(marca_num) , label
```

Esto creará una variable que tome los valores 1,2,...,N para los grupos que forma la varlist que le especifiquemos (en nuestro caso *marca*). La diferencia con **encode** es que nos permite agrupar por más de una variable a la vez.

En ambos casos, los valores numéricos que se asignan se determinan según el orden alfabético del texto original.

Contenido - Clase 2

1. Macros
2. Procesos repetitivos (loops)
3. Funciones
4. Conversión de tipos de variables
5. Bysort
6. Unión de bases de datos
7. Transformación de bases
8. Trabajo práctico

- Muchas funciones u operaciones requieren trabajar de manera independiente según grupos o categorías (ej: regiones, hogares, años).
- El comando **by** nos permite ejecutar un comando dentro de cada grupo definido por una o más variables.

```
[by varlist:] comando [varlist] [=exp] [if exp] [in rango] [weight] [ , opc]
```

```
by varlist: comando
```

Intuitivamente, cuando **by varlist:** aparece delante de otro comando lo que hace es ejecutarlo de manera independiente para cada grupo que forme de acuerdo a los valores que toman las variables especificadas en el **varlist**.

- Sin embargo, es un requisito tener ordenada la base de datos según la/s variable/s que se utiliza/n para definir los grupos.
- El prefijo **bysort** permite **ordenar** y luego ejecutar un comando **dentro de cada grupo**.
- Es equivalente a hacer **sort** y luego **by** en una sola línea.

```
sort varlist
by varlist : comando

* En una sola línea
bysort varlist : comando
```


Ejemplo 1

```
* Creamos una base de ejemplo
clear
input str1 grupo
"A"
"A"
"A"
"A"
"B"
"B"
"B"
end

* Generamos una variable con el orden dentro de cada grupo
bysort grupo: gen orden_en_grupo = _n

* Generamos una variable con la cantidad de observaciones de cada grupo
bysort grupo: gen cantidad_grupo = _N
```

Ejemplo 1

```
browse
```

Resultado:

```
grupo  orden_en_grupo  cantidad_grupo
A           1           4
A           2           4
A           3           4
A           4           4

B           1           3
B           2           3
B           3           3
```

Ejemplo 2

```
* Creamos una base de ejemplo
clear
input str1 grupo price
"A" 100
"A" 80
"A" 120
"B" 90
"B" 150
"B" 110
end

* Generamos una variable con el orden dentro de cada grupo según el precio
bysort grupo (price): gen orden_en_grupo = _n

* Generamos una variable con la cantidad de observaciones de cada grupo
bysort grupo: gen cantidad_grupo = _N
```

Ejemplo 2

```
browse
```

Resultado:

grupo	price	orden_en_grupo	cantidad_grupo
A	80	1	3
A	100	2	3
A	120	3	3
B	90	1	3
B	110	2	3
B	150	3	3

Contenido - Clase 2

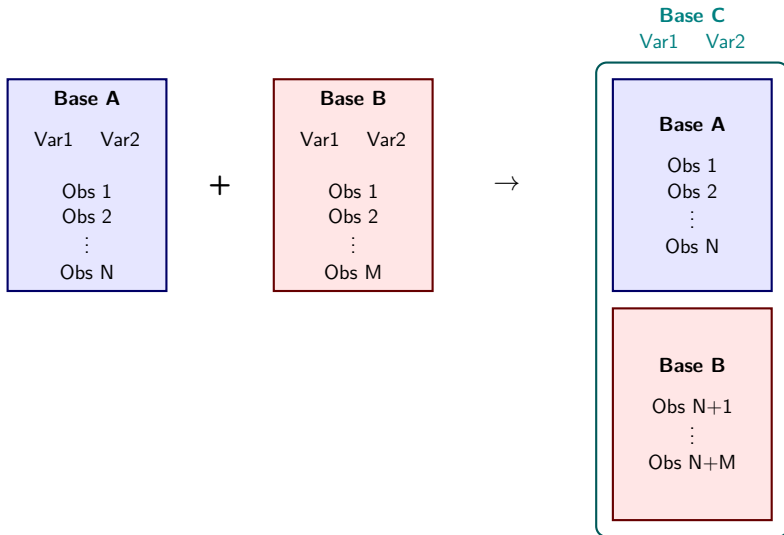
1. Macros
2. Procesos repetitivos (loops)
3. Funciones
4. Conversión de tipos de variables
5. Bysort
- 6. Unión de bases de datos**
7. Transformación de bases
8. Trabajo práctico

Podemos distinguir dos tipos de uniones:

1. Aquellas cuyo principal objetivo es agregar **nuevas observaciones** a la base (filas).
2. Aquellas que buscan agregar **nuevas variables** para las observaciones ya existentes en la base.

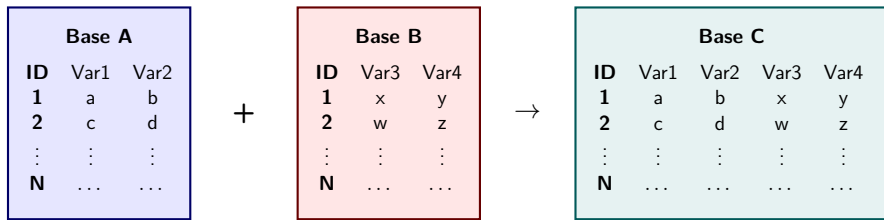
Si bien es posible que precisemos combinar ambos aspectos, para el primer tipo de unión lo más aconsejable es usar el comando **append**, mientras que para el segundo nos convendrá utilizar el comando **merge**.

Append



Unión de bases de datos

Merge



- **ID** es la variable que identifica a cada observación en ambas bases, y es la que se utiliza para realizar la unión (variable "llave").

Para ambos tipos de uniones nos referiremos como:

1. Base **master**: aquella que se encuentra cargada en Stata al momento de ejecutar la unión
2. Base **using**: aquella que se encuentra guardada en el disco y que se incorpora a la master al ejecutar la unión.

Append

Sintaxis:

```
append using filename [filename ...] [ , options]
```

- Con la base ***master*** cargada, ejecutamos esa sentencia especificando el nombre (y ruta) de la base ***using***.
- Notar que es posible unir más de una base al mismo tiempo.

Merge

El uso del merge es un poco más complicado ya que existen algunas variantes para especificar cómo deben unirse las filas:

- **1:1** (one-to-one): cada observación de la master se corresponde con una sola observación de la using.
- **1:m** (one-to-many): cada observación de la master se corresponde con una o más observaciones de la using.
- **m:1** (many-to-one): muchas observaciones de la master se asocian con una de la using.
- **m:m** (many-to-many): muchas observaciones de la master se asocian con muchas de la using.

Merge

Ejemplo sintaxis:

```
merge 1:1 varlist using filename [ , options]
```

- De esta forma se unirán la base *master* que se encuentre cargada con la base *using* que especifiquemos en *filename* utilizando como variable/s llave la/s que indiquemos en *varlist*.
- La/s variables llave debe/n ser del mismo tipo y aparecer en ambas bases con exactamente el mismo nombre.

Merge

Además, no solo nos reportará la cantidad de observaciones que ha logrado combinar, sino que también creará una variable con el nombre `_merge`, que tomará los siguientes valores:

Valor	Etiqueta	Descripción
1	Master	Observación que solo aparece en la base master
2	Using	Observación que solo aparece en la base using
3	Match	Observación matcheada que aparece en ambas bases

(Existen dos valores más que solo aparecen en caso de combinar bases con el objetivo de actualizar valores y se identifican con `Match_update` y `Match_conflict`)

Contenido - Clase 2

1. Macros
2. Procesos repetitivos (loops)
3. Funciones
4. Conversión de tipos de variables
5. Bysort
6. Unión de bases de datos
7. Transformación de bases
8. Trabajo práctico

- También es posible transformar la estructura de una base para organizar la misma información de manera diferente.
- El objetivo suele ser facilitar (o habilitar) análisis que requieren una estructura específica de los datos.
- Por ejemplo, para: generar y exportar tablas, construir gráficos o trabajar con datos de panel.

Preserve y restore

Cuando queremos hacer transformaciones “destructivas” para luego volver a la base original suele ser útil usar:

- **preserve** guarda una copia temporal del dataset tal como está en memoria.
- **restore** recupera exactamente esa versión guardada.

```
preserve
```

```
Comandos que transforman el dataset
```

```
...
```

```
restore
```


Preserve y restore

- **Ventaja principal:** permite volver al estado exacto del dataset sin reabrir archivos ni reejecutar todos los pasos previos.
- **Costo de memoria RAM:** al usar **preserve**, Stata guarda una copia del dataset en memoria; en bases grandes ese costo puede ser alto.
- **Limitación:** solo puede haber un **preserve** activo a la vez.
- **Recomendación:**
 1. Usar **preserve/restore**;
 2. Si la base es grande o la RAM es limitada, usar **tempfile**.

Collapse

Convierte el dataset en un conjunto de datos de promedios, sumas, u otros estadísticos.

Sintaxis:

```
collapse (clist) varlist [(clist)] [varlist] [if] [in] [weight] [ , options]
```

Collapse

Convierte el dataset en un conjunto de datos de promedios, sumas, u otros estadísticos.

Sintaxis:

```
collapse (clist) varlist [(clist)] [varlist] [if] [in] [weight] [ , options]
```

* Ejemplos:

```
collapse (mean) price trunk turn
```

```
collapse (mean) price (max) trunk (min) turn
```

Collapse

Y se vuelve incluso más útil cuando lo combinamos con la opción **by**.

* Ejemplo:

```
collapse (mean) price (max) trunk (min) turn , by(foreign)
```

El dataset pasaría a ser:

foreign	price	trunk	turn
Domestic	6072.42	23	31
Foreign	6384.68	16	32

- Si bien esta misma tabla la podríamos crear con **table**, de esta forma queda directamente en el dataset, lo que facilita su manipulación y uso.
- Pero también sirve obviamente para otras transformaciones que no tengan que ver con crear tablas estadísticas, por ejemplo, para convertir un dataset de datos individuales en uno de datos agregados a nivel de hogar, región, país, trimestre, año, etc.

Contract

Reemplaza el dataset por uno nuevo que consta de todas las combinaciones entre las variables que le especifiquemos y una nueva variable que contiene la frecuencia de cada combinación.

Sintaxis:

```
contract varlist [if] [in] [weight] [ , options]
```

Contract

Permite calcular directamente los porcentajes. Ejemplo:

```
contract foreign , percent(foreign_percent)
```

foreign	_freq	foreign_percent
Domestic	52	70.27
Foreign	22	29.73

Xpose

- Transpone los datos, transformando las variables en observaciones y las observaciones en variables.
- Todas las variables nuevas, es decir, las creadas por la transposición, se convierten en el tipo de almacenamiento predeterminado. Por lo tanto, cualquier variable original que fuera una cadena dará como resultado observaciones que contengan *missing* en el nuevo dataset.
- Si bien es reversible, se pierde el contenido de las variables de texto.

Transformación de bases

Xpose

```
xpose , clear
```

País	2025	2026
1	200	220
2	100	140
3	50	.



v1	v2	v3
1	2	3
200	100	50
220	140	.

Transformación de bases

Xpose

```
xpose , varname clear
```

País	y2025	y2026
1	200	220
2	100	140
3	50	.



v1	v2	v3	varname
1	2	3	País
200	100	50	y2025
220	140	.	y2026

Reshape

Base Long

i	j	stub
1	1	200
1	2	50
2	1	100
2	2	140
3	1	220

← Reshape →

Base Wide

i	stub ₁	stub ₂
1	200	50
2	100	140
3	220	.

Transformación de bases

Reshape - Ejemplo

Base Long

país	año	pbi
Bra	2010	200
Uru	2010	50
Arg	2010	100
Arg	2011	140
Bra	2011	220

← Reshape →

Base Wide

i	pbi 2010	pbi 2011
Bra	200	50
Arg	100	140
Uru	220	.

Transformación de bases

Reshape

Para ir de *long* a *wide*:

```
reshape wide stub , i(i) j(j)
```

Donde j es una variable ya existente

Base Long

i	j	stub
1	A	200
2	A	50
3	A	100
1	B	140
2	B	220



i	stub A	stub B
1	200	220
2	100	140
3	50	.

Base Wide

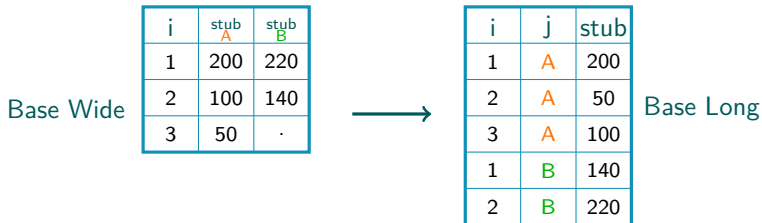
Transformación de bases

Reshape

Para ir de *wide* a *long*:

```
reshape long stub , i(i) j(j)
```

Donde j es una nueva variable



Contenido

1. Macros
2. Procesos repetitivos (loops)
3. Funciones
4. Conversión de tipos de variables
5. Bysort
6. Unión de bases de datos
7. Transformación de bases
8. Trabajo práctico

Trabajo Práctico 2

- **Objetivo:** Practicar organización e integración de bases, transformación de estructuras, construcción de indicadores, elaboración y exportación de tablas.
- **Bases:** *comercios.xlsx*, *precios.dta* y *productos.dta*.
- **Sugerencia:** Abrir cada archivo para familiarizarse con su estructura antes de empezar.
- **Importante:** El do-file debe correr de principio a fin. La única modificación requerida debe ser la definición de rutas al inicio.

Trabajo práctico 2

1. Importar ***comercios.xlsx***, hacer las modificaciones necesarias y guardarla como una base temporal.
2. Cargar ***precios.dta*** y unir con ***productos.dta***. Conservar solo observaciones con correspondencia en ambas bases.
3. Agregar la información de comercios (comercio y bandera). Conservar solo observaciones con *match*.

Trabajo práctico 2

4. Generar una variable numérica de identificación única por bandera llamada **store**.
5. Modificar la estructura para que el precio de cada año quede en una variable separada, siendo la unidad de análisis **producto-store**.
6. Crear una variable con la variación porcentual del precio entre 2025 y 2024.
7. Crear una variable indicadora que valga 1 si el producto subió de precio y 0 si no subió.

Trabajo práctico 2

8. Mostrar cuántos productos subieron y cuántos no subieron por **store**.
9. Mostrar qué productos mantuvieron el mismo precio entre 2024 y 2025.
10. Construir una tabla con promedio y desvío estándar de la variación de precio por **store**.
11. Construir tres tablas de variación promedio del precio por **store**, desagregadas por **rubro**, **categoría** y **subcategoría**.
12. Exportar las tres tablas a un único archivo **tp2.xlsx**, en pestañas **rubro**, **categoría** y **subcategoría**.
13. Resolver el punto anterior con un bucle.